SLE-4: Architecture Decision Record (ADR)

Graph Search System Using BFS and DFS

PRN	25UAM115
Name	Namrata Negarale
Division	B
Date	11/10/2026
Status	Accepted

Part A — Architecture Decision Record (ADR)

ADR-01: Use of a Visited Set in BFS and DFS

1. Context

The project is a Graph Search System using Breadth-First Search (BFS) and Depth-First Search (DFS) on a 15-node graph. The same graph is used to examine both search strategies. During traversal, a node may be reached through more than one path, so the algorithm needs a way to track nodes that have already been explored.

In this implementation, both BFS and DFS use a set named visited to record explored nodes.

2. Decision

I decided to use a visited set in both BFS and DFS. The algorithm checks whether a node has already been visited before processing it and records it in the set, helping avoid unnecessary repeated processing.

3. Alternatives Considered

Alternative 1: Do not use a visited set — rejected because the search could repeatedly process previously explored nodes.

Alternative 2: Use a list instead of a set — a list can work, but membership checks require scanning its elements; a set is suitable for visited-node checks.

Alternative 3: Use only the queue or stack — rejected because these structures manage pending nodes and do not replace a separate record of explored nodes.

4. Rationale / Justification

The visited set gives the search a clear record of explored nodes. It helps keep graph traversal controlled and avoids unnecessary repeated processing. A conceptual example is:

```
visited = set()
if current_node in visited:
    continue
visited.add(current_node)
```

The same general approach is used in BFS and DFS and is represented by the Visited/Memory element in the SLE-3 C4 architecture. It also supports the node-expansion counter. In the recorded run of this 15-node project, both BFS and DFS expanded 15 nodes. Node counts can vary with the graph, start node, goal node, and traversal order.

5. Consequences

Positive:

- Avoids unnecessary repeated processing of explored nodes.
- Makes traversal more controlled and easier to reason about.
- Provides a clear record of explored nodes and supports node-expansion counting.

- Fits naturally into the Visited/Memory part of the SLE-3 architecture.

Negative / trade-offs:

- Additional memory is required to store visited nodes.
- The set must be maintained separately for each search run.

Part B — Short Summary of My SLE Journey

SLE-1: I created a simple rule-based chatbot and learned the basics of building a small AI-related system and documenting the use of AI tools.

SLE-2: I implemented BFS and DFS on a 15-node graph and compared their behaviour using node-expansion counting, timeit benchmarking, and py-spy profiling.

SLE-3: I designed the C4 architecture for the same graph-search system, including the Input Module, Graph Management, Search Engine, Visited/Memory, and Output Module.

SLE-4: I selected the use of a visited set as a design decision and documented its context, alternatives, justification, and trade-offs in an Architecture Decision Record.

Part C — AI Contribution Note

AI tools were used to help clarify the assignment requirements, explain BFS and DFS concepts, and organize the ADR sections. I reviewed the content and am responsible for understanding the implementation and the design decision reported here.

Conclusion

This report documents the decision to use a visited set in BFS and DFS to track explored nodes. The decision is visible in the search implementation and is represented by the Visited/Memory element in the SLE-3 C4 architecture. It helps control traversal, avoids unnecessary repeated processing, and supports a meaningful comparison of the algorithms.